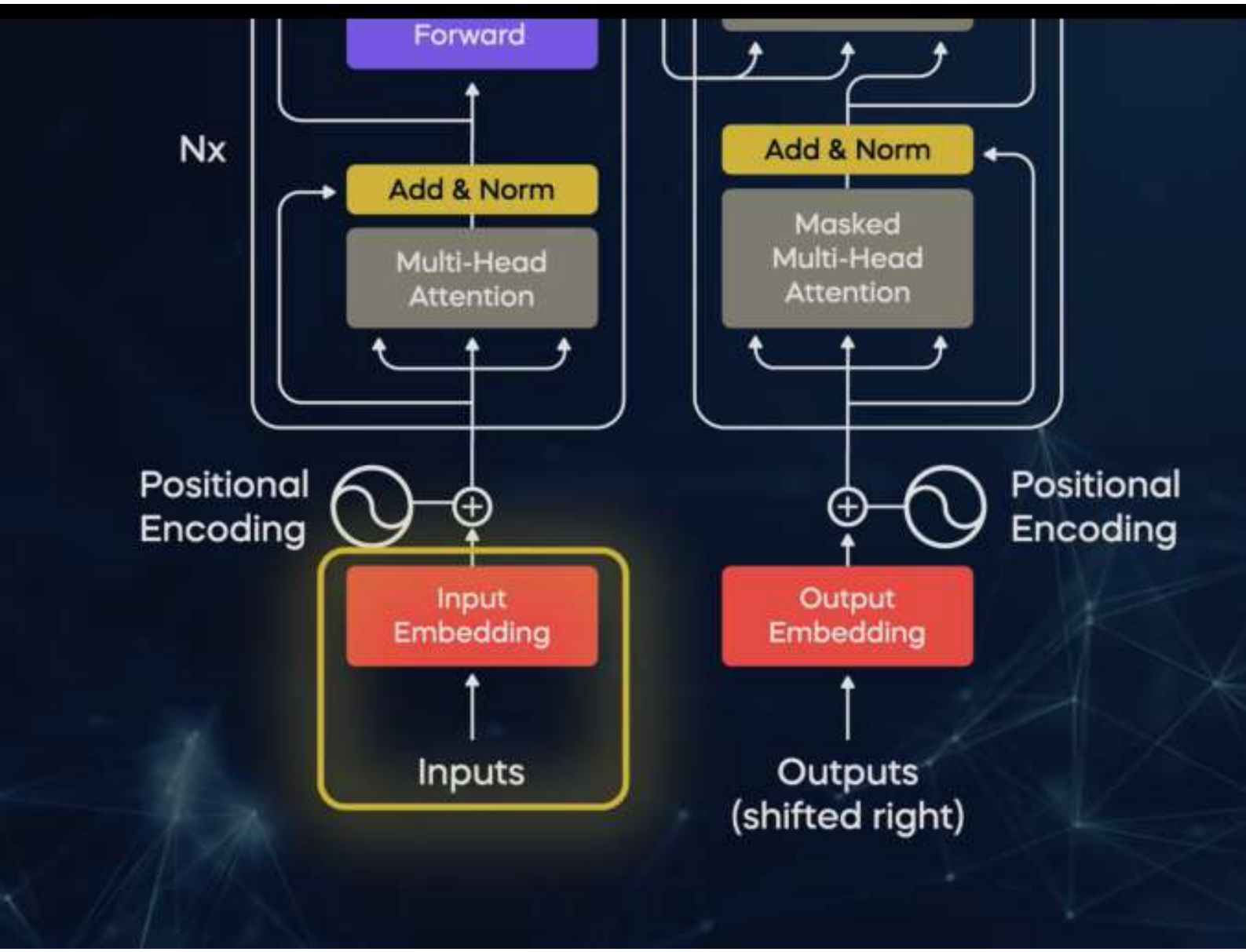

Output Probabilities
Softmax
Linear
Add & Norm
Feed Forward
Add & Norm
Multi-Head Attention
Add & Norm
Masked Multi-Head Attention
Nx
Add & Norm
Feed Forward
Add & Norm
Multi-Head Attention
Nx
Positional Encoding
Positional Encoding
Input Embedding
Output Embedding
Inputs
Outputs (shifted right)


Forward
Nx
Add & Norm
Multi-Head Attention
Add & Norm
Masked Multi-Head Attention
Positional Encoding
Positional Encoding
Input Embedding
Output Embedding
Inputs
Outputs (shifted right)

Creating the input embeddings

1. Break down the input into tokens

Tokens = Words, subwords or characters

"I love natural language processing"

["I", "love", "natural", "language", "processing"]

Creating the input embeddings

"I love natural language processing"

["I", "love", "natural", "language", "processing"]

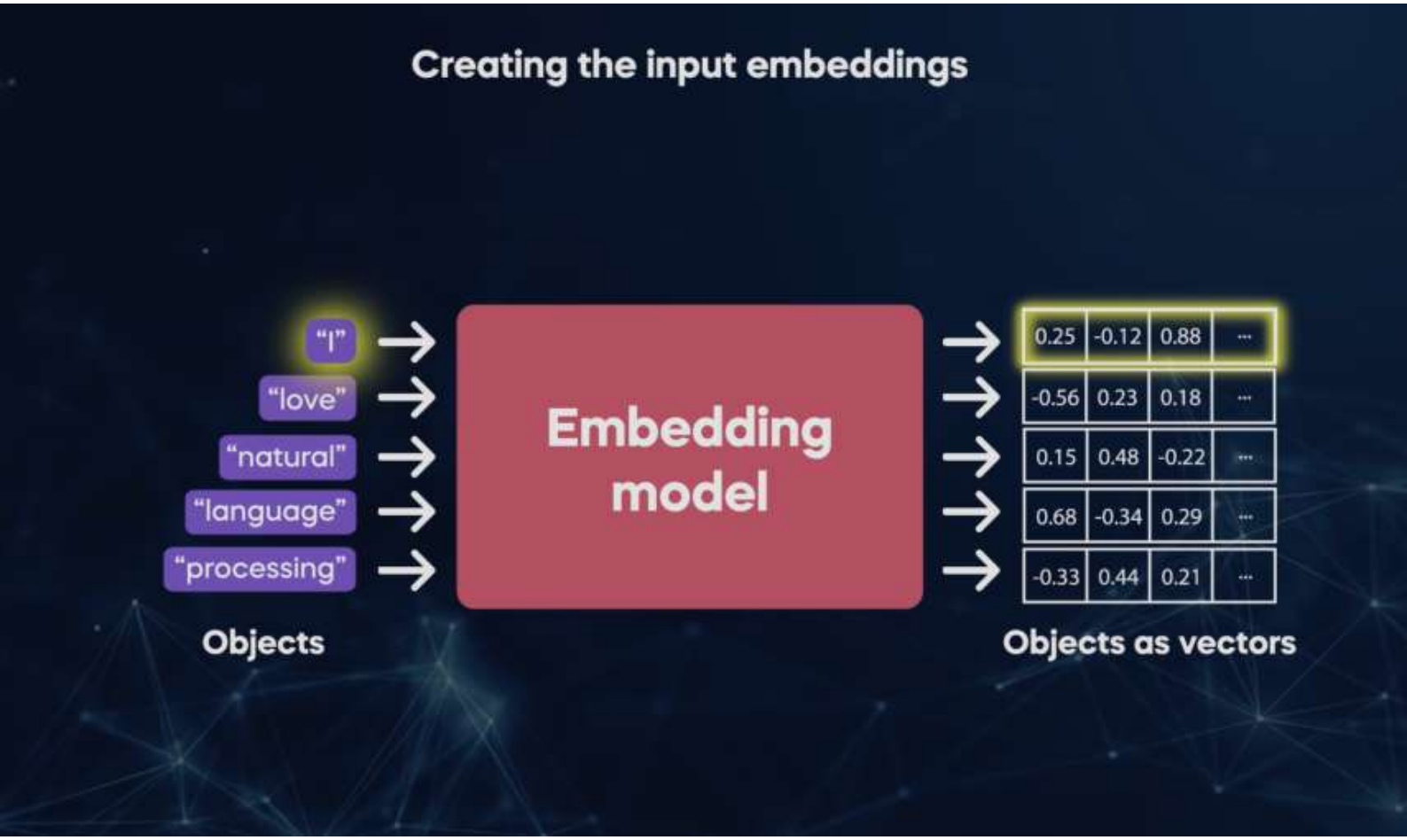
"I" - 34

"love" - 56

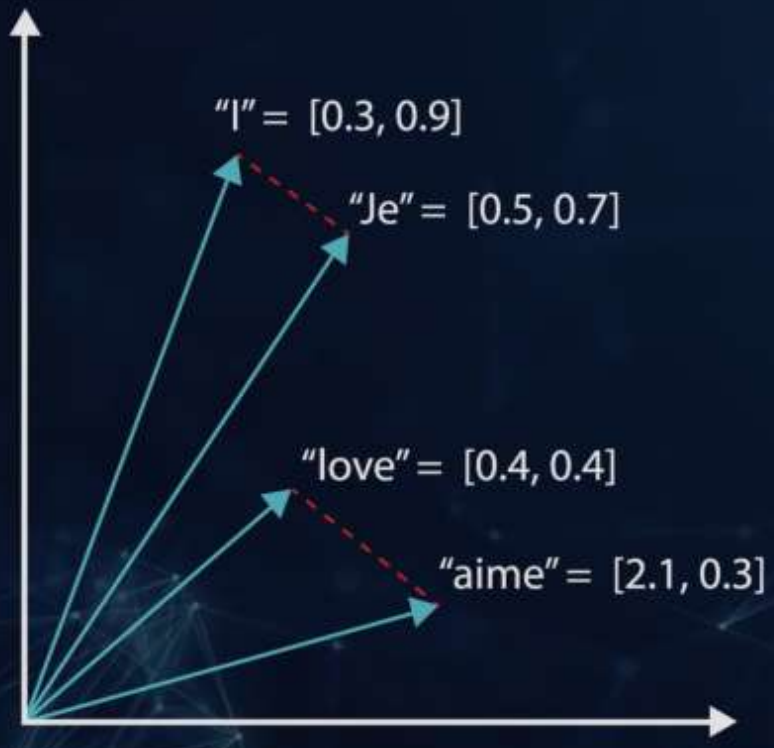
"natural" - 96

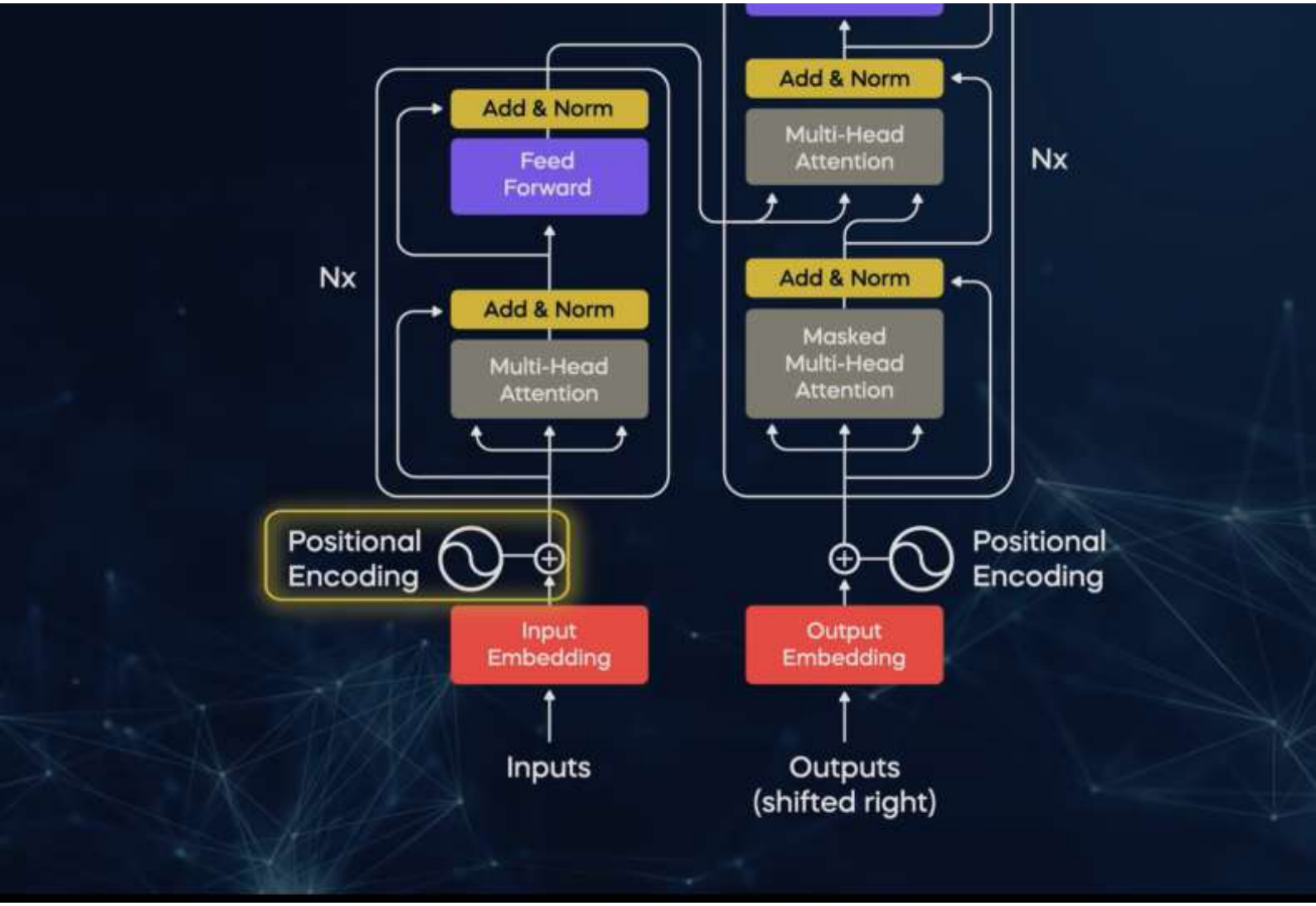
"language" - 12

"processing" - 24

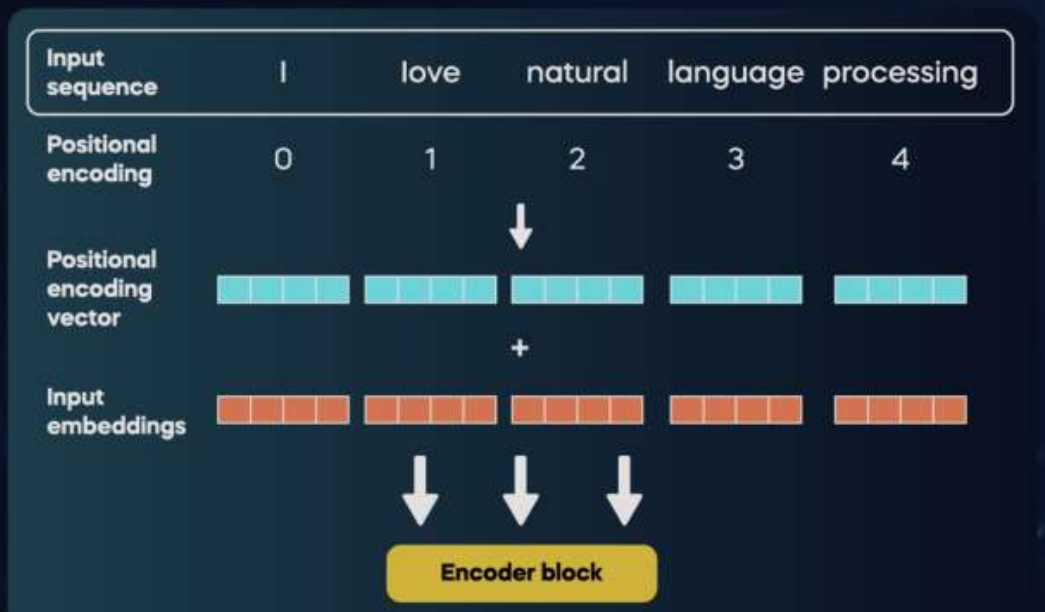

Creating the input embeddings
"I"
"love"
"natural"
"language"
"processing"
Objects
Embedding model
0.25 -0.12 0.88 ...
-0.56 0.23 0.18 ...
0.15 0.48 -0.22 ...
0.68 -0.34 0.29 ...
-0.33 0.44 0.21 ...
Objects as vectors

Creating the input embeddings




Add & Norm
Feed Forward
Add & Norm
Multi-Head Attention
Nx
Positional Encoding
Input Embedding
Inputs
Add & Norm
Multi-Head Attention
Add & Norm
Masked Multi-Head Attention
Nx
Positional Encoding
Output Embedding
Outputs (shifted right)

Positional Encoding

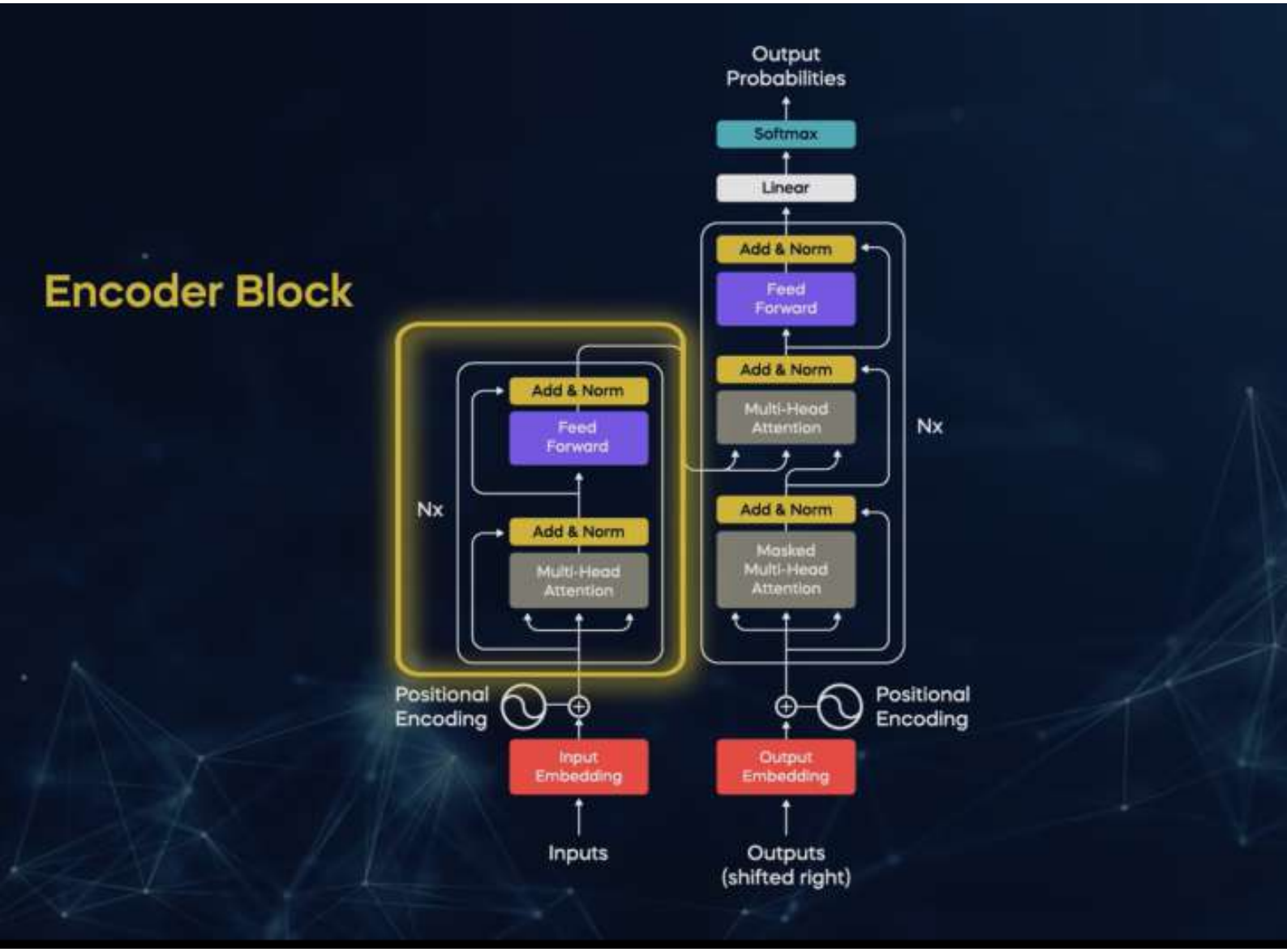


Padding

Adds special tokens or zeros to the embeddings of shorter sequences

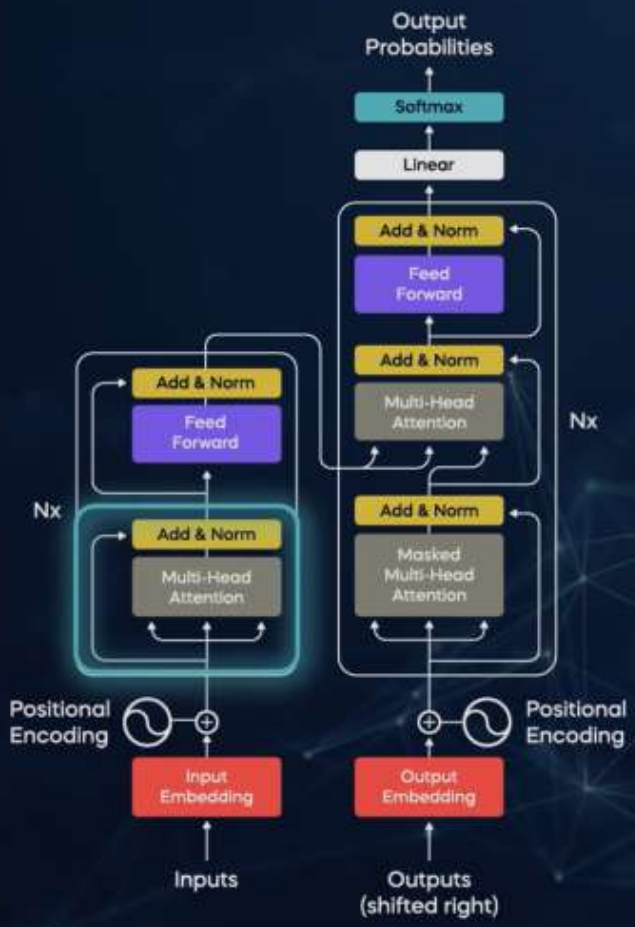
Truncation

Removes tokens from longer sequences


Encoder Block
Output Probabilities
Softmax
Linear
Add & Norm
Feed Forward
Add & Norm
Multi-Head Attention
Add & Norm
Masked Multi-Head Attention
Nx
Add & Norm
Feed Forward
Add & Norm
Multi-Head Attention
Nx
Positional Encoding
Positional Encoding
Input Embedding
Output Embedding
Inputs
Outputs (shifted right)

Multi-Head Attention Layer

- It allows the model to weigh the importance of different tokens
- Gives us attention vectors that capture contextual information between every token in the text



For each token in the input sequence, create three vectors:

Query Vector

Key Vectors

Value Vector

For each token in the input sequence, create three vectors:

Query Vector

This query vector represents the token's own "question" about the other tokens in the sequence

Input sequence:
A robot must obey the orders given to it

What is 'it' referring to?

For each token in the input sequence, create three vectors:

Key Vectors

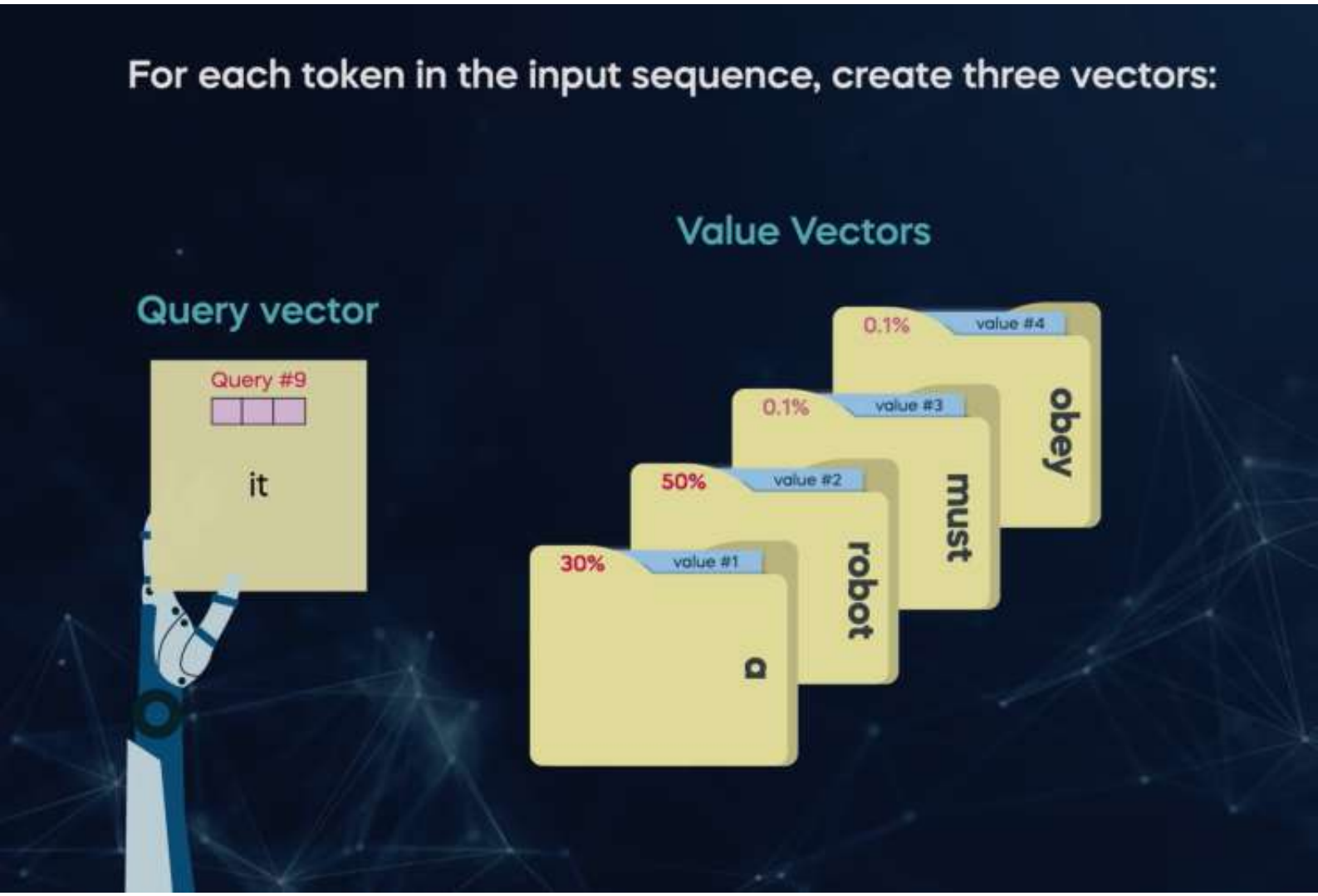
- Hold information about all the other tokens in the sequence.
- Each key vector corresponds to a different token and contains information.
- Help determine how much attention should be given to that token

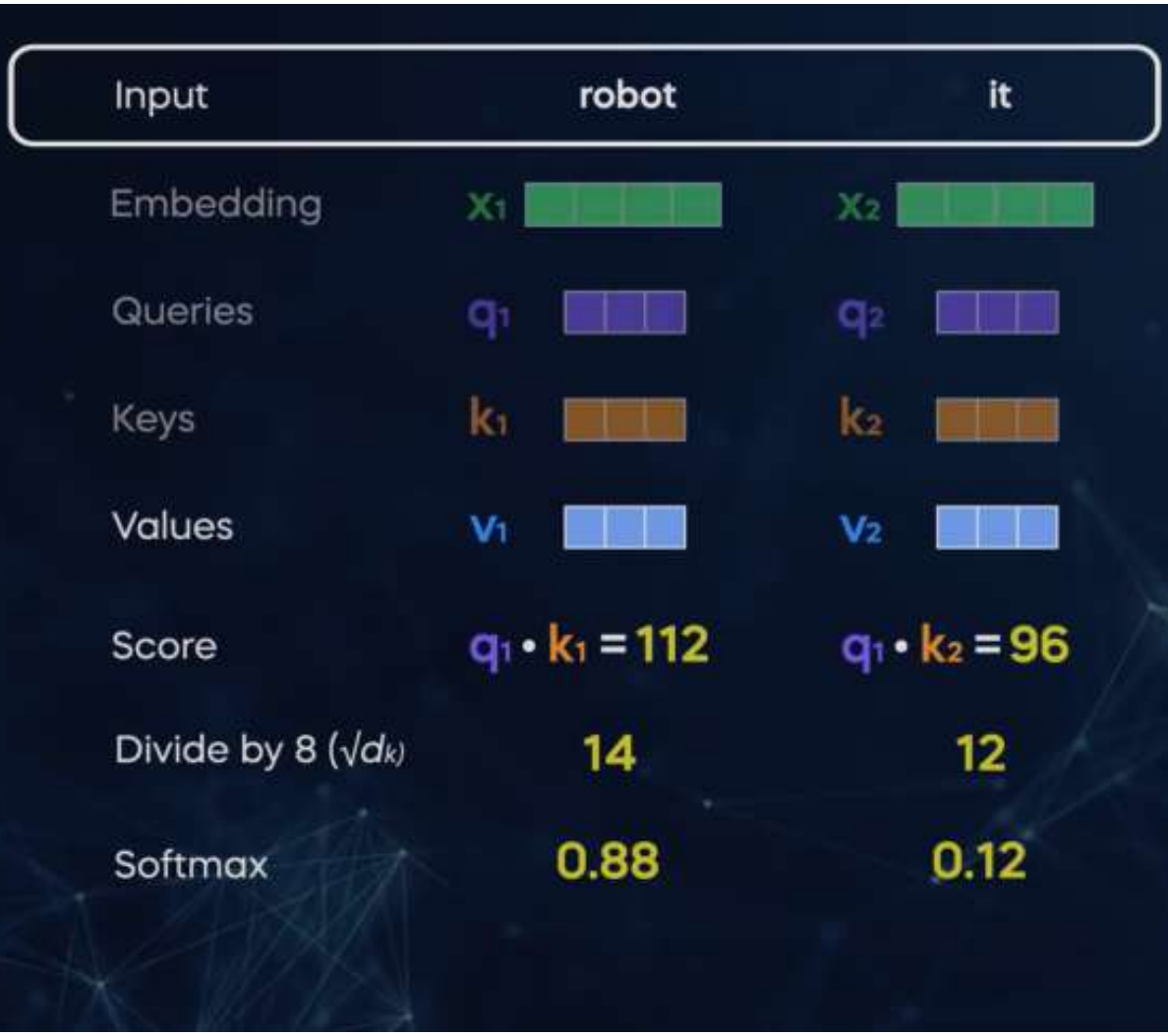
For each token in the input sequence, create three vectors:

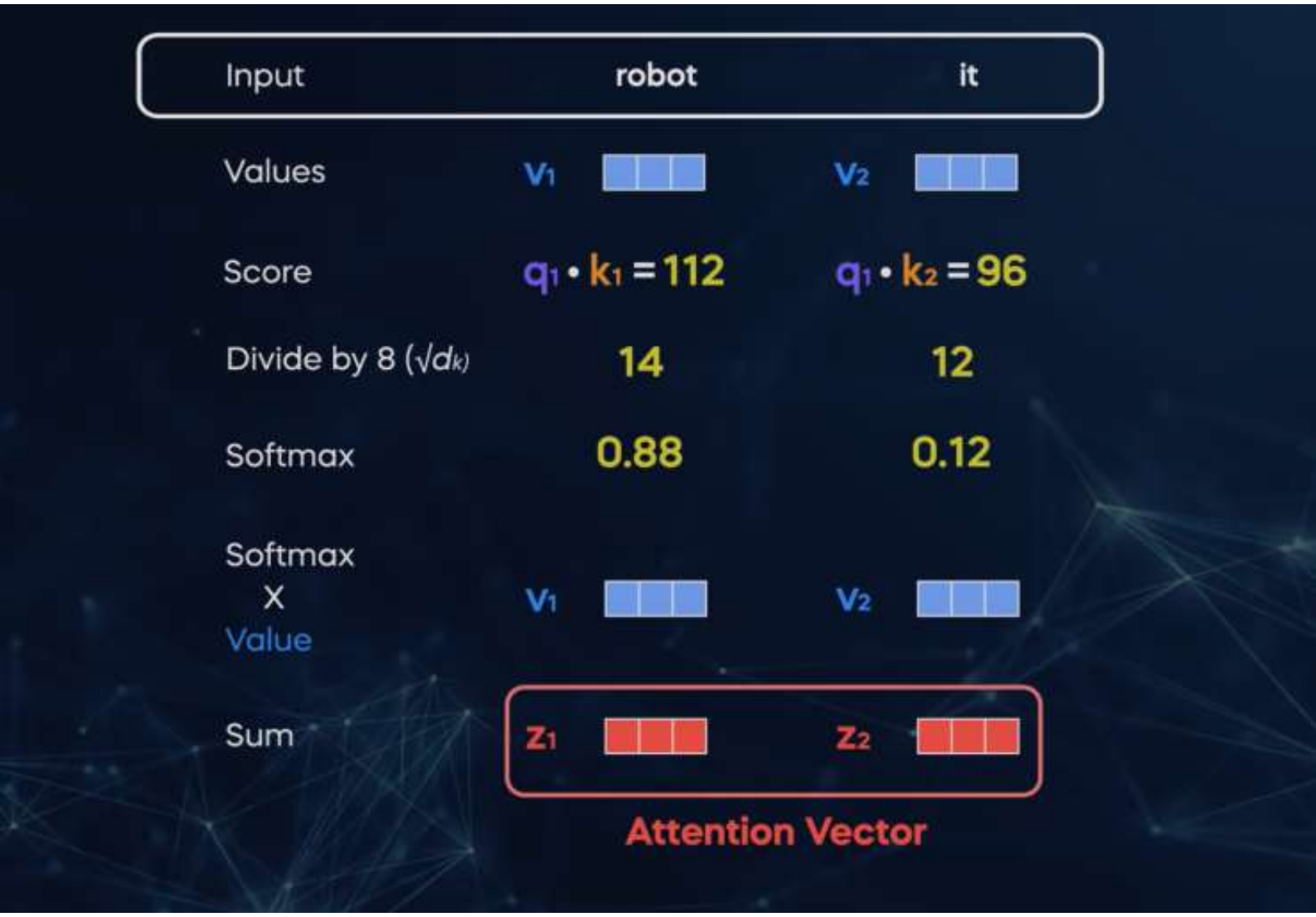
Value Vectors

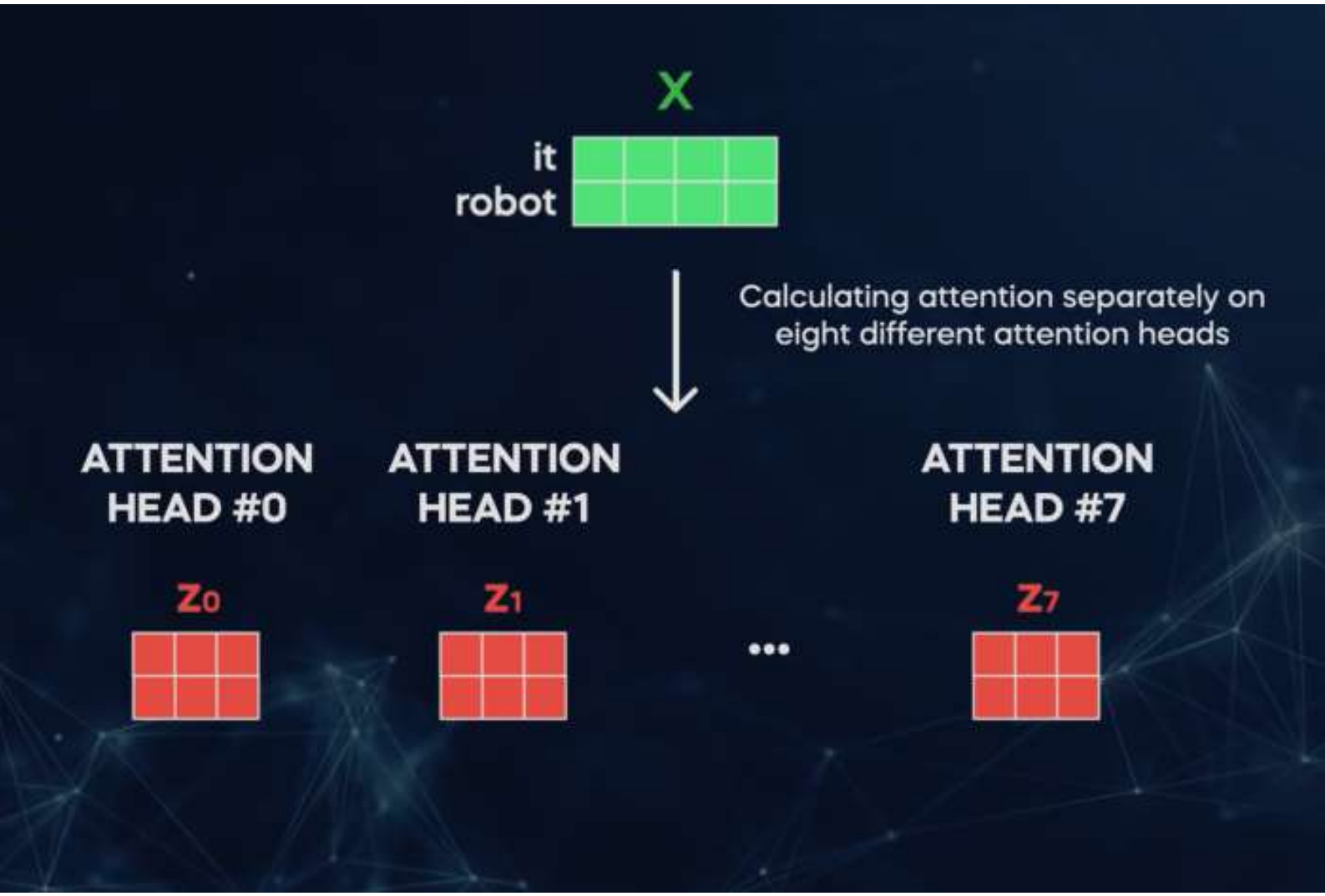
- Associated with each token in the input sequence
- "Information containers"

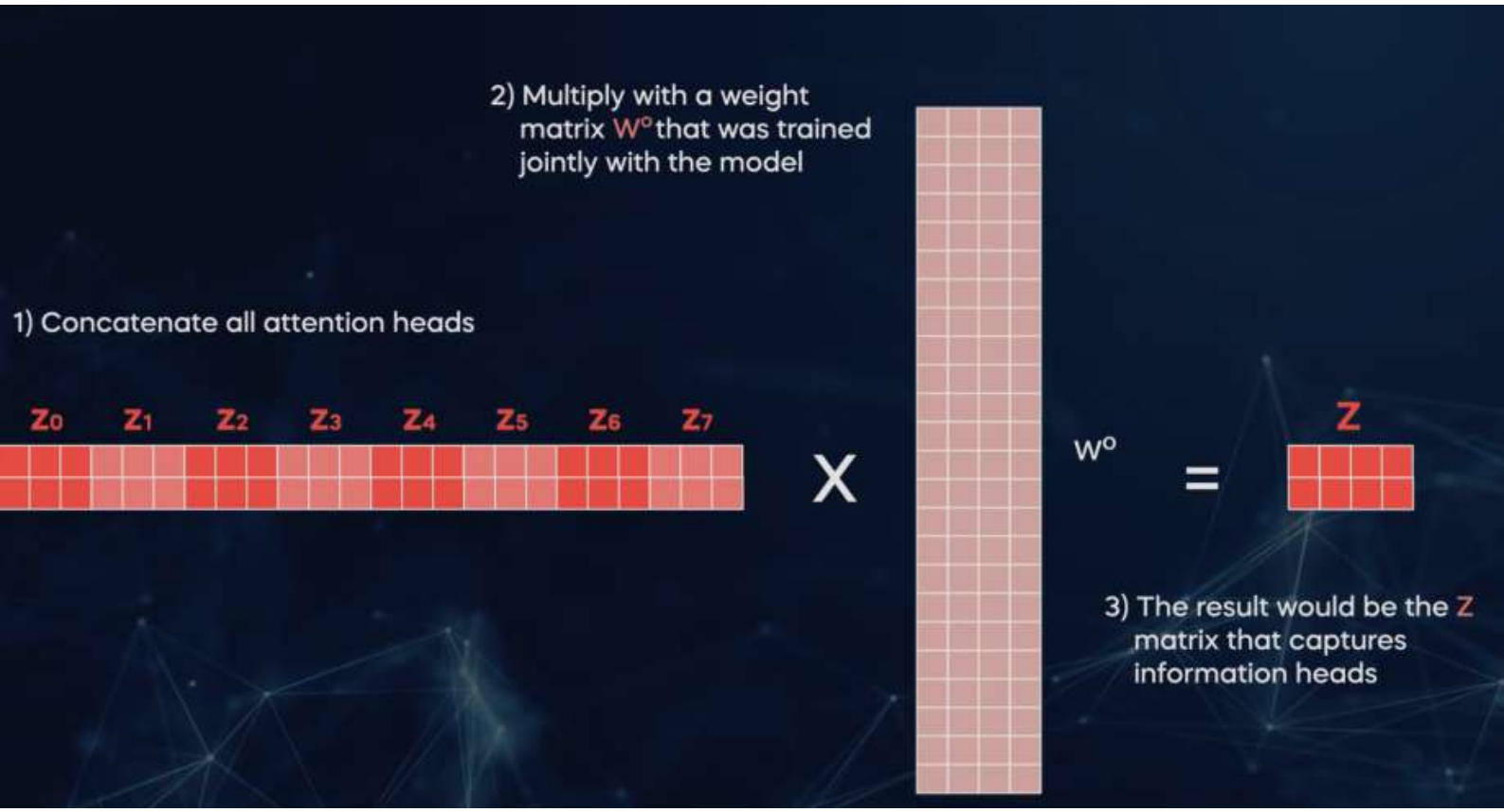
For each token in the input sequence, create three vectors:




Input
robot
it
Embedding
x_1
x_2
Queries
q_1
q_2
Keys
k_1
k_2
Values
v_1
v_2
Score
$q_1 \cdot k_1 = 112$
$q_1 \cdot k_2 = 96$
Divide by 8 ($\sqrt{d_k}$)
14
12
Softmax
0.88
0.12

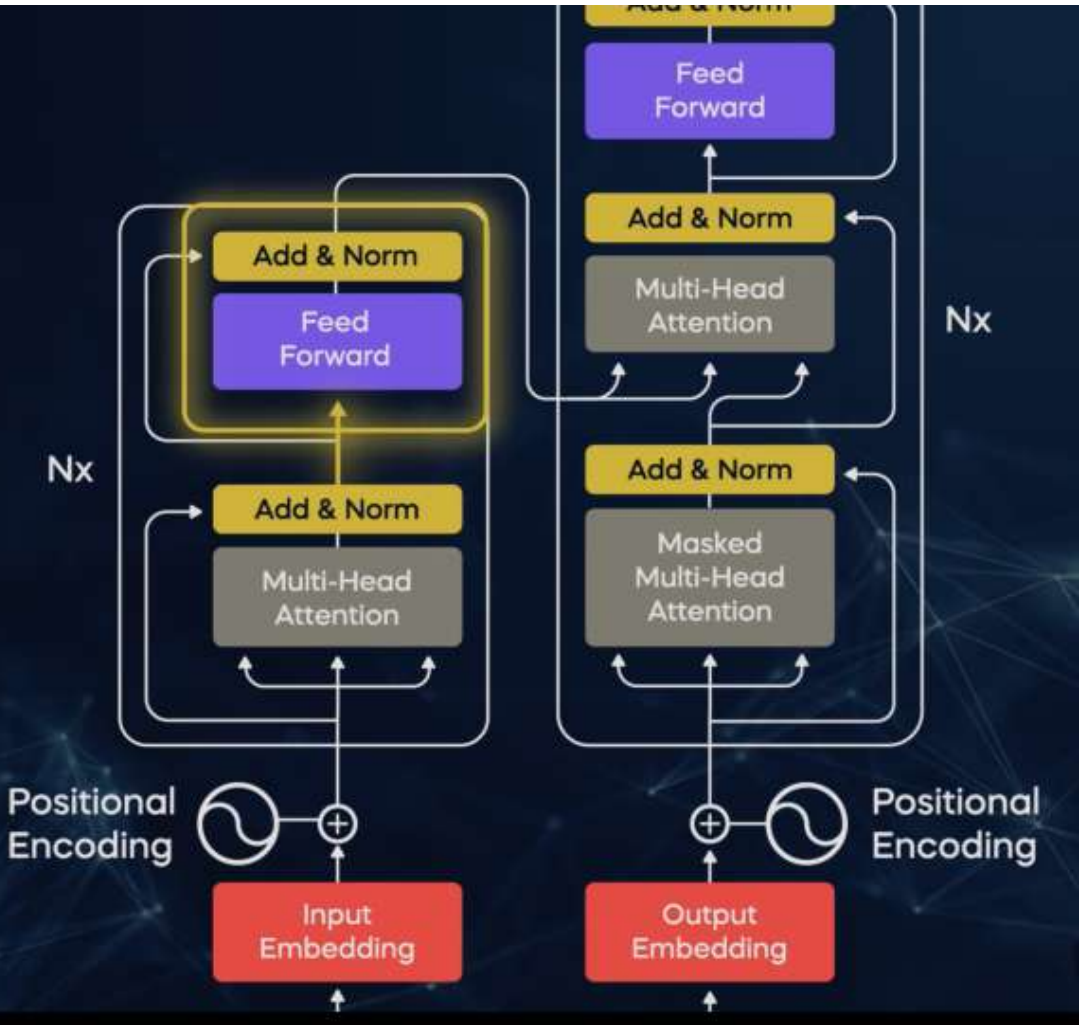

Input
robot
it
Values
v1
v2
Score
q1 • k1 = 112
q1 • k2 = 96
Divide by 8 (√dk)
14
12
Softmax
0.88
0.12
Softmax
X
Value
v1
v2
Sum
z1
z2
Attention Vector

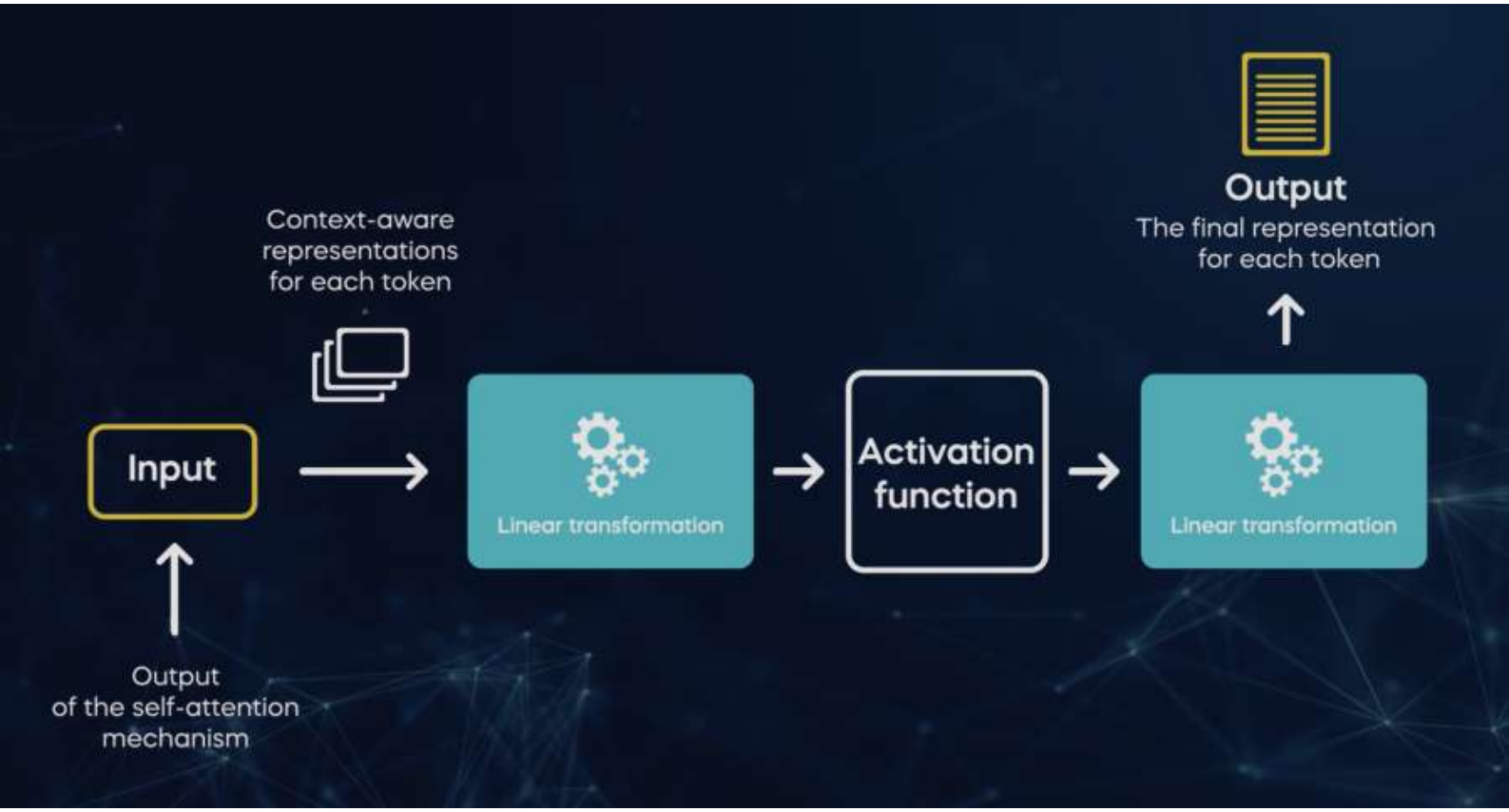

X
it
robot
Calculating attention separately on eight different attention heads
ATTENTION HEAD #0
ATTENTION HEAD #1
ATTENTION HEAD #7
Z0
Z1
Z7


2) Multiply with a weight matrix W° that was trained jointly with the model
1) Concatenate all attention heads
Z0 Z1 Z2 Z3 Z4 Z5 Z6 Z7
X
W°
=
Z
3) The result would be the Z matrix that captures information heads

Feed Forward Layer

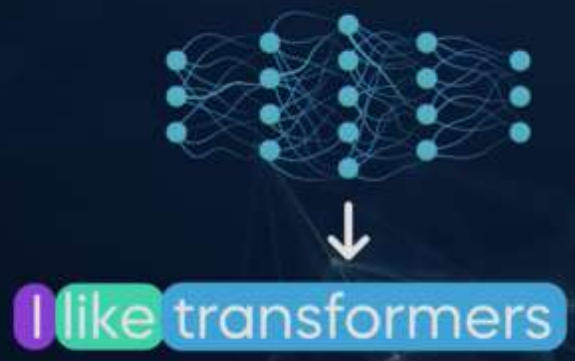
- Models complex, relationships within the input sequence
- Processes and transforms the information captured during the self-attention mechanism

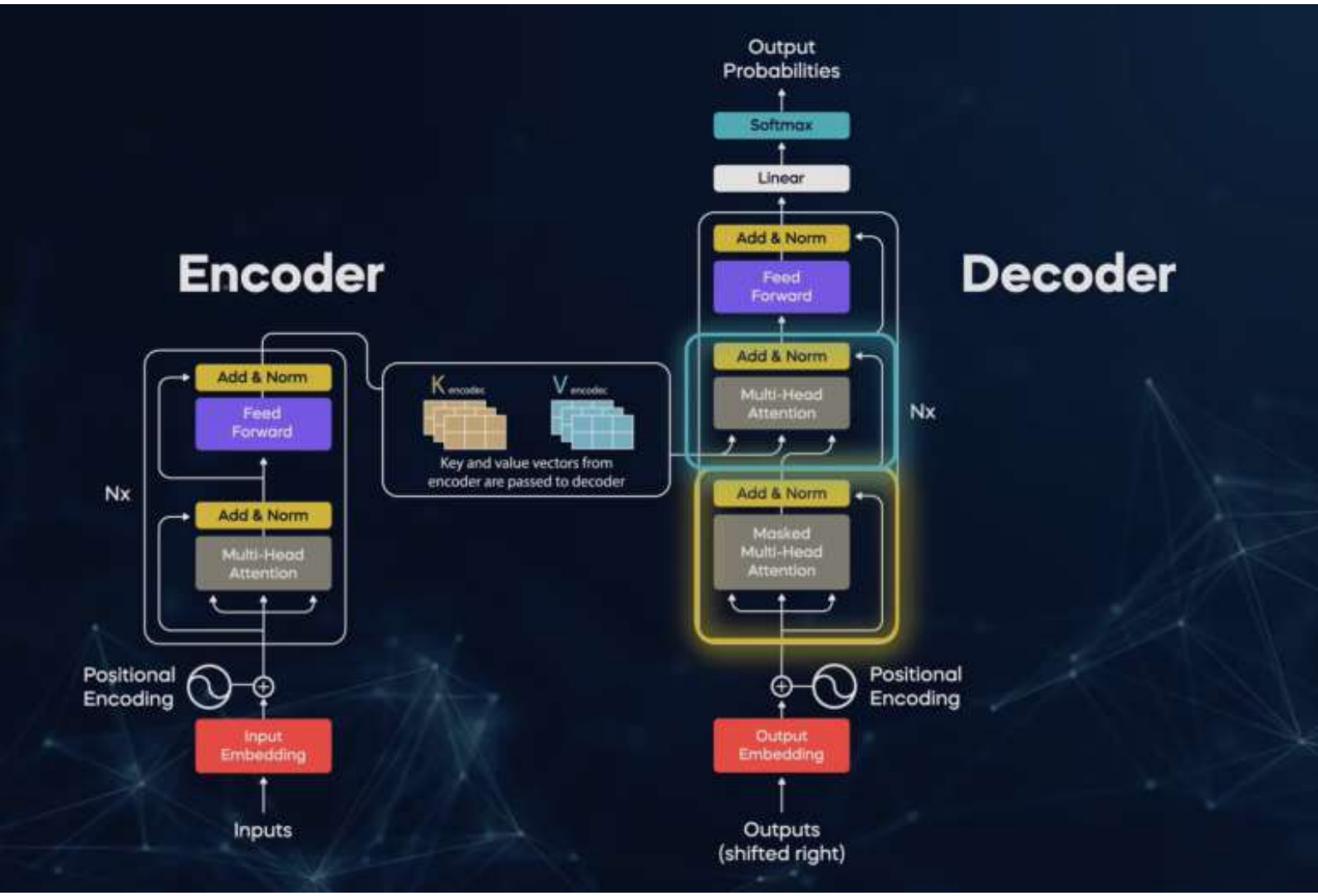


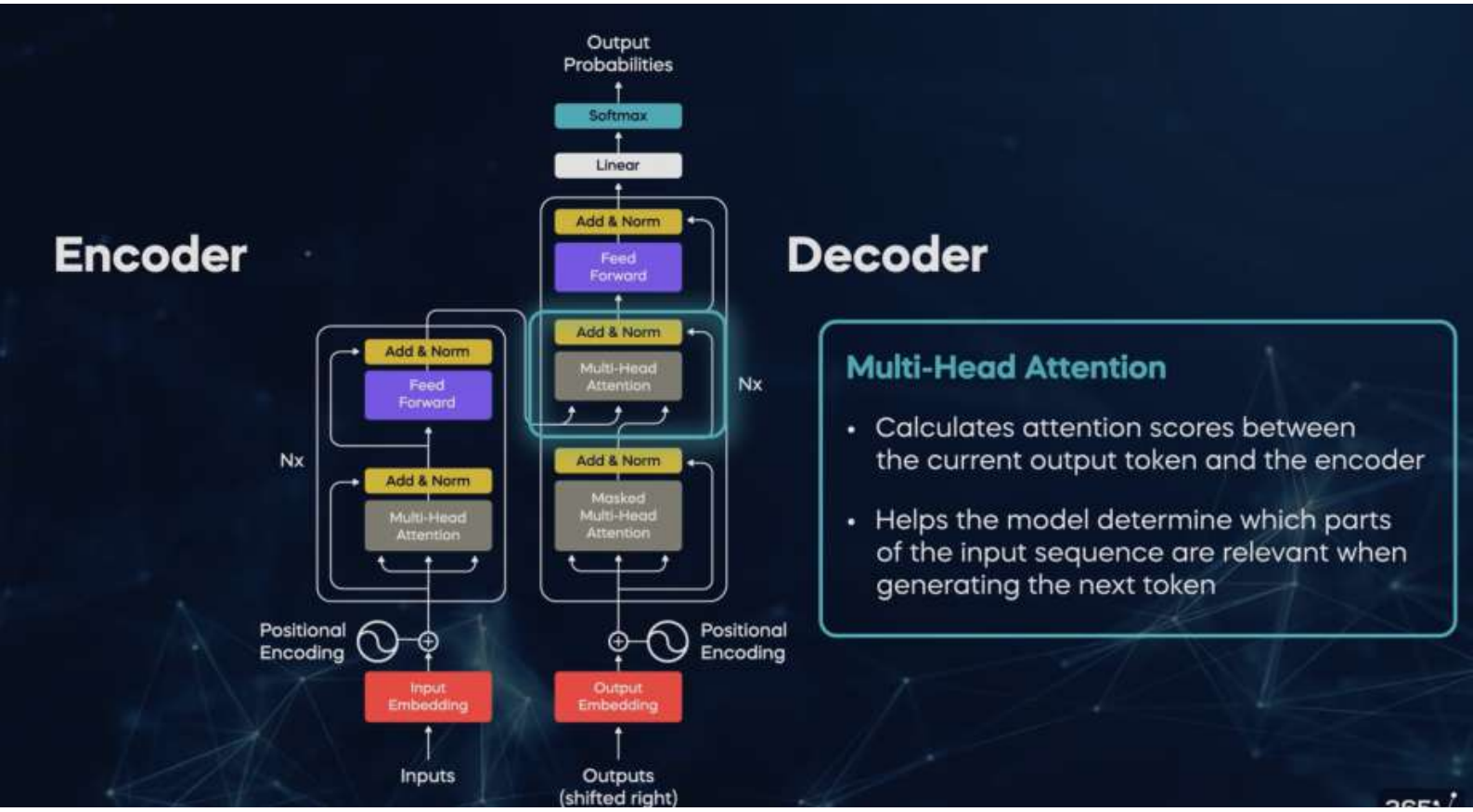

Output
The final representation
for each token
Context-aware
representations
for each token
Input
Linear transformation
Activation
function
Linear transformation
Output
of the self-attention
mechanism

Feedforward layer:

- Acts as a set of neural network operations applied independently to each token's representation
- Helps the Transformer model capture and encode complex, non-linear relationships between tokens in the input sequence
- Applied independently
- It can be run in parallel




Output Probabilities
Softmax
Linear
Add & Norm
Feed Forward
Add & Norm
Multi-Head Attention
Add & Norm
Masked Multi-Head Attention
Nx
Decoder
Encoder
Add & Norm
Feed Forward
Add & Norm
Multi-Head Attention
Nx
K encodec
V encodec
Key and value vectors from encoder are passed to decoder
Positional Encoding
Input Embedding
Inputs
Positional Encoding
Output Embedding
Outputs (shifted right)


Encoder
Decoder
Output Probabilities
Softmax
Linear
Add & Norm
Feed Forward
Add & Norm
Multi-Head Attention
Add & Norm
Masked Multi-Head Attention
Nx
Add & Norm
Feed Forward
Add & Norm
Multi-Head Attention
Nx
Positional Encoding
Positional Encoding
Input Embedding
Output Embedding
Inputs
Outputs (shifted right)
Multi-Head Attention
• Calculates attention scores between the current output token and the encoder
• Helps the model determine which parts of the input sequence are relevant when generating the next token